

MAERII Module 2: Market Demand Intelligence Engine

Technical Design Document v1.0

Document Classification: Technical Architecture & Design Review
Author: MAERII Engineering Team
Date: June 2026
Module: Market Demand Intelligence Engine (Module 2 of 6)
Codebase Reference: [mfp_scraper/market_scraper/](#)

1. Executive Summary

1.1 Purpose

Module 2 of the MAERII platform — the **Market Demand Intelligence Engine** — addresses a critical information gap in India's Minor Forest Produce (MFP) value chain: the absence of structured, data-driven market demand signals for 87 government-listed MFP commodities. These commodities — including wild honey, tamarind, lac, sal seeds, and mahua flowers — are harvested by approximately 100 million tribal forest-dwellers across India, yet their market dynamics remain largely undocumented in any systematic digital format.

This module constructs a **proxy demand estimation system** by programmatically scraping e-commerce marketplace data, classifying scraped products against the MFP taxonomy using Large Language Models, and computing composite demand scores from publicly observable market signals.

1.2 Role Within MAERII Architecture

MAERII is a six-module AI platform designed to provide end-to-end intelligence for the MFP sector:

Module	Name	Status
Module 1	MFP Data Enrichment Engine	Complete
Module 2	Market Demand Intelligence Engine	Complete
Module 3	Supply Chain Mapping	Planned
Module 4	Pricing Intelligence	Planned
Module 5	Policy & Regulation Tracker	Planned
Module 6	Recommendation Engine	Planned

Module 2 occupies a foundational position: its demand scores and market signals feed directly into Modules 4 (pricing optimization), 5 (policy impact assessment), and 6 (actionable recommendations for tribal cooperatives).

1.3 Inputs from Module 1

Module 2 consumes `data/enriched_mfp_data.json` produced by Module 1, which contains:

- **87 MFP items** with structured fields: `mfp_id`, `name`, `scientific_name`, `category`, `mfp` (Minimum Support Price), `unit`, `states`, `current_products`, `potential_products`, `artisan_types`, `confidence` scores
- Each item's `current_products` and `potential_products` arrays serve as the **search query seed vocabulary** for market scraping

1.4 Outputs Produced by Module 2

Output File	Description	Schema
<code>market_demand_data.json</code>	87 items with full <code>market_analysis</code> blocks	Per-MFP demand scores, trends, summaries
<code>market_products.json</code>	Flat array of all scraped products	Deduplicated by <code>product_id</code> , with classifications
<code>market_sources.json</code>	Provenance tracking	Query → URL → MFP mapping
<code>market_analysis_log.json</code>	Validation and completeness report	Per-item quality metrics
<code>data/market_runs/</code>	Run manifests, previews, backups	Reproducibility artifacts

2. Architecture Overview

2.1 End-to-End Workflow

The pipeline follows a **Search** → **Fetch** → **Classify** → **Score** → **Persist** architecture, directly inheriting Module 1's proven pattern of **Search** → **Fetch** → **Extract** → **Validate** → **Write**.

The processing flow proceeds through the following stages in sequence:

Stage	Component	Input	Output	Description
1. Load	<code>market_pipeline.py</code>	<code>enriched_mfp_data.json</code> (87 items)	List of MFP dicts	Reads Module 1 output; filters by item name, resume index, or limit
2. Query Build	<code>ecommerce_client.py</code>	MFP name, <code>current_products</code> , <code>potential_products</code>	Up to 5 search strings	Generates India-targeted shopping

Stage	Component	Input	Output	Description
				queries per MFP
3. Fetch	<code>ecommerce_client.py</code>	Search queries	Raw product list	Calls Serper Shopping API; falls back to web search if empty
4. Normalize	<code>market_models.py</code>	Raw API responses	Standardized product dicts	Cleans prices, ratings, URLs; extracts attributes; assigns deterministic IDs
5. Classify	<code>market_extractor.py</code>	Normalized products + MFP taxonomy	Products with <code>matched_mfp_ids</code> and <code>confidence</code>	Keyword pre-filter → LLM classification (Groq → Gemini fallback)
6. Analyze	<code>market_extractor.py</code>	Classified products + MFP metadata	Competitor insights, demand drivers	LLM generates structured market analysis
7. Score	<code>market_models.py</code>	Matched products + MSP	Demand score (0–1), trend label	Weighted composite score from 5 market signals
8. Validate	<code>market_validator.py</code>	Complete market analysis block	Completeness score, issues, warnings	Checks schema, ranges, duplicates, coverage
9. Persist	<code>rollback.py</code>	Validated analysis	JSON files on disk	Atomic write with backup

Stage	Component	Input	Output	Description
				snapshots; run manifest logged

2.2 Layered Architecture

The module is organized into four distinct layers, each with clear boundaries:

Layer	Files	Responsibility	Rules
Interface Layer	main_market.py, __main__.py	CLI argument parsing, command routing, output formatting	Zero business logic. Never imports service or model layer directly for processing.
Orchestration Layer	market_pipeline.py, market_validator.py	Workflow coordination, state management, batch processing, atomic persistence	Owns the execution flow. May call any lower layer. Manages preview vs. apply modes.
Service Layer	ecommerce_client.py, market_extractor.py	External API communication (Serper, Groq, Gemini), rate limiting, retry logic	Each service is independently testable. No cross-service dependencies.
Data Layer	market_models.py, config_market.py	Pure functions (scoring, hashing, normalization, merging), constants, configuration	No I/O, no side effects, no external calls. Fully deterministic and unit- testable.

3. Technology Selection Rationale

3.1 Serper Shopping API

What Problem It Solves

The fundamental challenge is obtaining structured e-commerce product data (titles, prices, ratings, review counts, sellers) for MFP-derived products across Indian marketplaces — without violating platform Terms of Service or maintaining fragile scraping infrastructure.

Why Serper Was Chosen

Serper provides a **Google Shopping SERP proxy** that returns structured JSON from Google's Shopping tab, which aggregates product listings from Amazon.in, Flipkart, Meesho, JioMart, and hundreds of smaller Indian e-commerce sites in a single API call.

Alternatives Considered

Solution	Cost (monthly)	Structured Data	Multi-Marketplace	Maintenance	Legal Risk
Serper Shopping	\$50 (50K queries)	Native JSON	Google aggregates all	Zero	None (API-based)
Direct Amazon Scraping	\$0 (custom)	HTML parsing	Amazon only	High (DOM changes)	ToS violation
Direct Flipkart Scraping	\$0 (custom)	HTML parsing	Flipkart only	High	ToS violation
DataForSEO	\$100+	JSON	Multi-marketplace	Low	None
BrightData	\$500+	JSON	Multi-marketplace	Low	None
Oxylabs	\$300+	JSON	Multi-marketplace	Low	None
Amazon PA-API	Free (with affiliate)	JSON	Amazon only	Low	None

Why Serper Is Superior For This Project

1. **Module 1 already uses Serper** for web search — reusing the same API key eliminates credential management overhead and consolidates billing
2. **Google Shopping aggregation** provides the broadest marketplace coverage from a single query — products from Amazon.in, Flipkart, Meesho, and niche MFP e-commerce sites all appear in one response
3. **Cost efficiency:** At \$50/month for 50,000 queries, the budget supports **87 MFPs × 5 queries = 435 queries** per full run, leaving 99% of the quota for iterative development and re-runs
4. **India-specific targeting:** The **gl: "in"** (geolocation) parameter ensures results are specific to the Indian market, which is critical for MSP-relative pricing analysis

Limitations

- **No historical data:** Serper returns current listings only; no time-series pricing or review count history
- **Google Shopping coverage gaps:** Some niche MFP products (e.g., sal seed oil, mahua flowers) may have limited Google Shopping presence
- **No direct sales volume data:** Review counts and ratings are proxies, not actual sales figures

Future Improvements

- **DataForSEO integration** for historical SERP data and Amazon-specific product analytics
- **Direct marketplace API integration** (Amazon PA-API) for authoritative pricing and availability data
- **Periodic crawl scheduling** to build longitudinal datasets for trend analysis

3.2 Gemini Models (Primary LLM)

Why Gemini 2.5 Flash Is Primary

The `market_extractor.py` uses Gemini models (configured via `config.GEMINI_MODELS`) as a primary or fallback LLM provider for two critical tasks: **product classification** (matching scraped products to MFP categories) and **market analysis** (generating competitor insights and demand indicators).

Selection Criteria

Criterion	Gemini 2.5 Flash	GPT-4o	Claude Sonnet 4	Claude Opus 4	Llama 3.3 70B (Groq)	Mixtral 8x7B (Groq)
JSON Reliability	Native <code>response_mime_type: "application/json"</code>	JSON mode	No native JSON mode	No native JSON mode	Groq JSON mode	Variable
Classification Accuracy	Good	Excellent	Excellent	Excellent	Good	Moderate
Cost per 1M tokens (input)	\$0.15	\$2.50	\$3.00	\$10.00	Free (rate-limited)	Free (rate-limited)
Cost per 1M tokens (output)	\$0.60	\$10.00	\$15.00	\$30.00	Free (rate-limited)	Free (rate-limited)
Latency (avg)	~1.5s	~2.5s	~3.0s	~5.0s	~0.8s	~0.6s
Context Window	1M tokens	128K tokens	200K tokens	200K tokens	128K tokens	32K tokens
Free Tier	15 RPM				30 RPM	30 RPM
India Data Quality	Good	Good	Moderate	Good	Moderate	Moderate

Key Justifications

- Native JSON output:** Gemini's `response_mime_type: "application/json"` parameter guarantees valid JSON responses without prompt engineering hacks. For a pipeline processing 87 items in batch, JSON parse failures are catastrophic — Gemini eliminates this class of failure entirely.
- Cost efficiency:** At \$0.15/1M input tokens, a full run classifying products for 87 MFPs costs approximately \$0.02–\$0.05 total. This is **50–200× cheaper** than GPT-4o for equivalent accuracy on a structured classification task.
- 1M token context window:** While not strictly necessary for our current prompt sizes (~2K tokens per classification), the large context window provides headroom for future enhancements where we might want to classify entire product catalogs in a single prompt.

4. **Free tier availability:** Gemini's free tier (15 RPM) is sufficient for development, testing, and moderate production use of 87 MFP items.

Recommendation

Gemini 2.5 Flash is the optimal choice for this specific project. The task — structured classification and summarization — does not require the reasoning depth of GPT-4o or Claude Opus. The native JSON mode, cost efficiency, and free tier make it the clear winner for a batch classification pipeline where reliability and cost matter more than peak intelligence.

3.3 Groq Fallback

Why a Fallback Exists

LLM API services experience three categories of failure:

1. **Rate limiting (HTTP 429):** Temporary throttling during burst traffic
2. **Quota exhaustion:** Monthly or daily usage caps exceeded
3. **Service outages:** Infrastructure failures, maintenance windows

A pipeline processing 87 items over 30–60 minutes cannot tolerate a single-provider dependency. The **MarketExtractor** implements a **cascading fallback**: Groq → Gemini, with per-model exhaustion tracking.

Why Groq Is Suitable as Fallback

- **Zero cost:** Groq's free tier provides 30 requests per minute with no monthly cap
- **Ultra-low latency:** Groq's custom LPU hardware delivers ~0.6–0.8s response times, making it the fastest inference provider available
- **JSON mode support:** Groq's API supports `response_format: {"type": "json_object"}`, ensuring structured output
- **Model variety:** Access to Llama 3.3 70B and Mixtral models provides multiple fallback paths within a single provider

Reliability Considerations

Groq's free tier is rate-limited to 30 RPM and 6,000 tokens per minute. For our pipeline's 4-second inter-call delay, this translates to ~15 RPM effective throughput — well within Groq's limits. The **exhausted_models** tracking set ensures that once a model's quota is hit, the pipeline immediately moves to the next model rather than retrying indefinitely.

4. Market Intelligence Methodology

4.1 Why Market Demand Must Be Estimated

Direct sales data for MFP commodities is structurally unavailable for several reasons:

1. **No centralized sales registry:** Unlike organized retail, MFP transactions occur through fragmented channels — tribal cooperatives, local mandis, middlemen, and direct forest-to-consumer sales

- 2. **Government MSP data is input-only:** TRIFED (Tribal Cooperative Marketing Development Federation of India) publishes Minimum Support Prices but not actual transaction volumes
- 3. **E-commerce platforms protect sales data:** Amazon, Flipkart, and other marketplaces do not expose unit sales figures through any public API
- 4. **Informal economy dominance:** An estimated 70–80% of MFP trade occurs in informal markets with no digital footprint

This structural data vacuum necessitates a **proxy-based demand estimation approach** using publicly observable marketplace signals.

4.2 Academic Justification

The methodology draws from established research in **revealed preference theory** (Samuelson, 1938) and **digital trace analytics** (Golder & Macy, 2014):

- **Review volume** as a demand proxy is validated by Chevalier & Mayzlin (2006), who demonstrated a strong positive correlation between review counts and sales rank on Amazon
- **Product variety** as a market maturity indicator follows Hotelling's spatial competition model (1929) — more product variants indicate higher market demand supporting differentiated offerings
- **Rating quality** as market validation follows the quality-signaling literature (Spence, 1973) — sustained high ratings in competitive markets indicate genuine consumer satisfaction
- **Seller diversity** as a competition indicator draws from industrial organization theory — more sellers entering a market signals profitable demand
- **Price premium over MSP** captures value-addition potential — the gap between raw material MSP and finished product retail price indicates the economic opportunity for tribal producers

4.3 Individual Metric Analysis

Review Volume (Weight: 30%)

Aspect	Detail
What it measures	Aggregate number of consumer reviews across all matched products
Why it matters	Reviews are the strongest publicly available proxy for actual purchase volume. Amazon's internal data shows ~1–5% of buyers leave reviews, making review counts a reliable order-of-magnitude sales estimator.
Normalization	Log-scale: $\log_{1p}(\text{total_reviews}) / \log_{1p}(10000)$. Logarithmic scaling prevents products with viral review counts (e.g., Dabur Honey with 50,000+ reviews) from dominating the score.
Strengths	Directly correlated with purchase behavior; available across all e-commerce platforms; resistant to manipulation at scale
Weaknesses	Review rates vary by category (electronics ~3%, FMCG ~1%); incentivized reviews inflate counts; new products have zero reviews despite strong sales

Product Variety (Weight: 20%)

Aspect	Detail
What it measures	Count of unique products found on e-commerce platforms for this MFP
Why it matters	A market with 50 different honey products indicates higher demand than one with 3. Product proliferation is a lagging indicator of sustained demand.
Normalization	Linear: $\min(\text{product_count} / 50, 1.0)$. Cap at 50 reflects the practical upper bound for niche MFP product categories.
Strengths	Easy to measure; robust across marketplaces; captures market breadth
Weaknesses	Conflates SKU fragmentation (same product, different sizes) with genuine variety; biased toward commodities with easier manufacturing

Rating Quality (Weight: 20%)

Aspect	Detail
What it measures	Average star rating across matched products, normalized to [0, 1]
Why it matters	High average ratings in a competitive market indicate genuine product quality and consumer satisfaction. For MFP products, high ratings suggest successful value-addition and packaging.
Normalization	Direct: $\text{mean}(\text{ratings}) / 5.0$
Strengths	Universal metric across all platforms; captures quality perception; penalizes categories with quality issues
Weaknesses	Rating inflation (4.0+ averages are common); survivorship bias (poorly rated products get delisted); doesn't capture rating trajectory

Price Premium (Weight: 15%)

Aspect	Detail
What it measures	Ratio of average retail price to MSP (Minimum Support Price), indicating value-addition potential
Why it matters	If Wild Honey MSP is ₹225/kg but packaged honey retails at ₹1,140 avg, the 5.1× markup reveals substantial value-addition opportunity for tribal producers.
Normalization	$\min(\text{avg_price} / (\text{msp} \times 10), 1.0)$. A 10× MSP multiplier represents the practical ceiling for premium MFP products.
Strengths	Directly measures economic opportunity; leverages TRIFED's authoritative MSP data; unique to this project
Weaknesses	MSP reflects raw material cost, not processing cost; packaging and branding costs are not subtracted; MSP may not reflect actual procurement prices

Seller Diversity (Weight: 15%)

Aspect	Detail
What it measures	Count of unique sellers/brands offering products in this MFP category
Why it matters	Multiple sellers competing in a category signals profitable demand. A market with one seller might be a monopoly; a market with 15 sellers is clearly viable.
Normalization	Linear: <code>min(unique_sellers / 20, 1.0)</code> . Cap at 20 reflects the observation that most Indian e-commerce MFP categories plateau at 15–25 active sellers.
Strengths	Captures competitive dynamics; indicates market accessibility; identifies monopolistic vs. competitive categories
Weaknesses	Same company may operate multiple seller accounts; aggregator sellers distort counts; does not capture market share distribution

5. Demand Score Design

5.1 Current Implementation

The demand score is computed in `calculate_demand_score()` as a weighted linear combination:

Mathematical Formula

The demand score is a weighted linear combination of five normalized market signals:

Demand Score (D) = $W_r \times S_r + W_q \times S_q + W_v \times S_v + W_p \times S_p + W_s \times S_s$

Where each signal is normalized to the range [0.0, 1.0] as follows:

Signal	Symbol	Description	Weight	Normalization Formula
Review Volume	S_r	Total review count across matched products	$W_r = 0.30$	$S_r = \min(\ln(1 + R) / \ln(10001), 1.0)$ where R = total reviews
Rating Quality	S_q	Average star rating across products	$W_q = 0.20$	$S_q = \text{mean_rating} / 5.0$
Product Variety	S_v	Count of unique matched products	$W_v = 0.20$	$S_v = \min(\text{product_count} / 50, 1.0)$
Price Premium	S_p	Avg retail price relative to MSP	$W_p = 0.15$	$S_p = \min(\text{avg_price} / (\text{MSP} \times 10), 1.0)$
Seller Diversity	S_s	Count of unique sellers/brands	$W_s = 0.15$	$S_s = \min(\text{unique_sellers} / 20, 1.0)$

Constraints:

- All weights sum to 1.0: $W_r + W_q + W_v + W_p + W_s = 1.0$
- Final score is clamped: $0.0 \leq D \leq 1.0$

5.2 Critical Evaluation

The current weights (30/20/20/15/15) are reasonable but not optimal. A critical analysis reveals:

Issue 1: Review Volume Should Be Higher

Review volume is the strongest empirical proxy for actual purchase volume (Chevalier & Mayzlin, 2006). The current 30% weight is conservative. In practice, an MFP item with 10,000+ reviews and mediocre ratings has far stronger demand than one with perfect ratings but 50 reviews.

Issue 2: Rating Quality Is Over-Weighted

Rating quality at 20% introduces a bias toward mature, well-established product categories. MFP products that are newly commercialized may have few reviews but excellent ratings from early adopters — this should not inflate demand scores disproportionately.

Issue 3: Seller Diversity and Price Premium Are Under-Weighted

Seller diversity (15%) is a strong structural indicator of market viability. Multiple independent sellers entering a market is a leading indicator of demand — often preceding review volume growth.

5.3 Recommended Weights

Based on market analytics principles and the specific characteristics of the MFP sector:

Metric	Current Weight	Recommended Weight	Justification
Review Volume	0.30 (30%)	0.35 (35%)	Strongest demand proxy; increase to reflect empirical correlation with sales
Product Variety	0.20 (20%)	0.25 (25%)	Market breadth is highly informative for niche MFP categories
Rating Quality	0.20 (20%)	0.15 (15%)	Reduce to avoid mature-product bias; ratings are a quality signal, not a demand signal
Seller Diversity	0.15 (15%)	0.15 (15%)	Appropriate — competitive dynamics are an important but secondary indicator
Price Premium	0.15 (15%)	0.10 (10%)	MSP-relative pricing is informative but noisy; reduce to minimize outlier sensitivity

Impact Analysis

With the recommended weights, the formula becomes:

$$D(\text{new}) = 0.35 \times Sr + 0.15 \times Sq + 0.25 \times Sv + 0.10 \times Sp + 0.15 \times Ss$$

5.4 Worked Example: Wild Honey

Using the actual pipeline output from the test run:

Raw Data:

- Total reviews: 12,777
- Average rating: 4.43
- Products found: 50
- Average price: ₹1,139.79
- MSP: ₹225/kg
- Unique sellers (estimated): ~18

Score Computation (Current Weights):

Metric	Raw Value	Normalized Score	Weight	Contribution
Review Volume	12,777	$\ln(12778)/\ln(10001) = 0.998$	0.30	0.299
Rating Quality	4.43	$4.43/5.0 = 0.886$	0.20	0.177
Product Variety	50	$50/50 = 1.000$	0.20	0.200
Price Premium	₹1,140 vs ₹225	$1140/2250 = 0.507$	0.15	0.076
Seller Diversity	18	$18/20 = 0.900$	0.15	0.135
Total				0.887

Actual output: 0.903 (minor variance from estimation due to exact seller count)

This score correctly identifies Wild Honey as a **high-demand** MFP commodity, which aligns with industry knowledge — honey is the most commercialized MFP in India.

6. Trend Classification Methodology

6.1 Current Implementation

Trend classification in `calculate_trend()` uses a composite engagement signal:

Engagement

Quality

Signal

= min(ln(1 + R) / ln(5001), 1.0)

= mean_rating / 5.0

= 0.6 × Engagement + 0.4 × Quality

where R = total reviews

The signal is then classified using fixed thresholds:

Classification	Condition
Rising	Signal ≥ 0.65
Stable	$0.35 \leq \text{Signal} < 0.65$
Declining	Signal < 0.35

6.2 Critical Evaluation

Strengths

- **Simple and interpretable:** Fixed thresholds are easy to explain to non-technical stakeholders
- **Deterministic:** Same input always produces same output, enabling reproducibility
- **Computationally trivial:** No statistical distribution fitting required

Weaknesses

- **Not truly temporal:** The current implementation uses cross-sectional data (a snapshot) to infer trends. Without historical data, "trend" is actually "market engagement level" — an important distinction.
- **Fixed thresholds are arbitrary:** The 0.65/0.35 boundaries have no empirical basis. They may misclassify items in a dataset where most scores cluster around 0.5–0.7.
- **No relative ranking:** A commodity with Signal = 0.66 is "rising" while one with 0.64 is "stable" — a meaningless distinction.

6.3 Alternative Approaches

Method	Description	Pros	Cons	Suitability (n=87)
Fixed Thresholds (current)	Absolute cutoffs	Simple, interpretable	Arbitrary, no distribution awareness	Acceptable
Percentile Ranking	Top 33% = rising, middle 33% = stable, bottom 33% = declining	Distribution-aware, self-calibrating	Requires full dataset, always produces equal-sized groups	Recommended
Quantile Ranking	Quartile-based classification	Finer granularity	Over-segments small datasets	Marginal
Z-Score	Standard deviations from mean	Statistically rigorous	Assumes normality; difficult to explain to non-technical stakeholders	Over-engineered
Market Basket	Relative to category peers	Contextually meaningful	Requires category hierarchy	Insufficient data

6.4 Recommendation

For a dataset of 87 MFPs, **percentile ranking** is the statistically optimal approach:

```
# Proposed implementation
import numpy as np

def classify_trend_percentile(scores: list[float], item_score: float) -> str:
    p33 = np.percentile(scores, 33)
    p67 = np.percentile(scores, 67)
    if item_score >= p67:
        return "rising"
    elif item_score >= p33:
        return "stable"
    else:
        return "declining"
```

Justification: Percentile ranking is self-calibrating — it adapts to the actual distribution of MFP engagement signals rather than imposing arbitrary boundaries. For $n=87$, each tercile contains ~29 items, providing statistically meaningful groupings.

However, the current fixed-threshold approach is retained in v1.0 for two pragmatic reasons:

1. **Incremental processing:** Percentile ranking requires the full dataset to be processed before any single item can be classified. The current pipeline supports single-item runs (`--item "Wild Honey"`), which would be impossible with percentile ranking.
2. **Stability:** Adding or removing items would shift all percentile boundaries, potentially reclassifying unchanged items — a confusing UX for end users.

The recommended migration path is to implement percentile ranking as a **post-processing step** after all 87 items are analyzed, while retaining fixed thresholds for single-item preview runs.

7. Search Strategy Justification

7.1 Current Strategy

For each of the 87 MFP items, the `build_search_queries()` method generates up to **5 search queries**:

1. "{MFP Name} India buy online" — Primary discovery query
2. "{Current Product 1} {MFP Name} India" — Top current product
3. "{Current Product 2} {MFP Name} India" — Second current product
4. "{Potential Product 1} India buy" — Top potential product
5. "{Potential Product 2} India buy" — Second potential product

Example: Wild Honey (MFP ID: 2)

Query #	Search Query	Intent
1	"Wild Honey India buy online"	Discover the primary market for wild honey
2	"Raw Honey Wild Honey India"	Find existing raw honey products
3	"Liquid honey Wild Honey India"	Find liquid honey variants
4	"Royal Jelly India buy"	Explore potential high-value product
5	"Propolis India buy"	Explore bee-derived health product market

7.2 Why This Strategy

Coverage Analysis

The 5-query strategy provides three tiers of market coverage:

- 1. **Broad discovery** (Query 1): Captures the general market landscape — any product associated with this MFP name
- 2. **Current market** (Queries 2–3): Targets products that tribal producers already create, providing the most relevant pricing and demand data
- 3. **Potential market** (Queries 4–5): Explores adjacent product categories that represent future value-addition opportunities

API Cost Analysis

Scale	Queries	Serper Cost	Cost per MFP
87 MFPs × 5 queries	435	\$0.44	\$0.005
87 MFPs × 10 queries	870	\$0.87	\$0.010
87 MFPs × all products	~1,740	\$1.74	\$0.020

At \$50/month for 50,000 queries, the 5-query strategy uses only **0.87%** of the monthly quota per full run, leaving ample room for iterative development.

Recall vs. Precision

Strategy	Recall	Precision	API Cost	Noise
1 query (MFP name only)	Low	High	Minimal	Low
3 queries (name + 2 current)	Medium	High	Low	Low
5 queries (name + 2 current + 2 potential)	High	Medium	Low	Medium
All products (5–10 queries)	Very High	Low	Medium	High
Exhaustive (all products + variations)	Maximum	Low	High	Very High

The 5-query strategy optimizes the **recall-to-noise ratio**: it captures both existing and potential markets while keeping noise manageable through downstream LLM classification.

7.3 Better Alternatives

For future versions, two improvements are recommended:

1. **Adaptive query count:** MFPs with 0 results from 5 queries should automatically expand to include additional product names or synonym variations
2. **Query quality scoring:** Track which queries produce the most matched products per MFP and prioritize them in subsequent runs

8. LLM Classification Strategy

8.1 Why AI Classification Is Required

The fundamental challenge is **semantic product-to-category matching** across a diverse taxonomy. Consider the product title:

"Himalayan Raw Forest Honey - 100% Pure Wild Multiflora Honey - 500g Glass Jar"

A human immediately recognizes this as matching MFP #2 (Wild Honey). But the mapping requires understanding that:

- "Himalayan" is a geographic qualifier, not a separate product category
- "Raw Forest" and "Wild" are synonyms for the MFP's harvesting method
- "Multiflora" is a honey variety descriptor
- "500g Glass Jar" is packaging, not a product category
- The product matches "Wild Honey" despite neither word appearing adjacently

This level of semantic understanding is beyond keyword matching but well within LLM capabilities.

8.2 Why Keyword Matching Is Insufficient

The `_keyword_classify()` method in `market_extractor.py` serves as a **first-pass filter and fallback**, not the primary classifier. Its limitations are structural:

False Positives

Product Title	Keyword Match	Correct MFP	Issue
"Honey Mustard Sauce 200ml"	"honey" → Wild Honey	None (condiment)	Substring match on unrelated product
"Wild Berry Jam 350g"	"wild" → Wild Honey	None (berry jam)	"Wild" is ambiguous
"Pure Silk Cocoon Facial Brush"	"silk", "cocoon" → Tasar Silk	None (cosmetic tool)	Material reference ≠ product match

False Negatives

Product Title	Keyword Match	Correct MFP	Issue
"Artisanal Forest-Harvested Raw Honey"	None (no exact keyword match)	Wild Honey	Paraphrased terminology
"Jungle Beeswax Candle - Natural"	None	Wild Honey (beeswax)	Derived product not in keyword list
"Tribal Women's Cooperative Mahua Spirit"	None	Mahua Flowers	Product name uses local terminology

The LLM classifier resolves both categories by understanding semantic relationships, product context, and the MFP taxonomy simultaneously.

8.3 Classification Examples

Input: "Himalayan Raw Forest Honey 500g"

Keyword classifier: Matches "honey", "raw", "forest" → confidence 0.7
LLM classifier: Matches MFP #2 (Wild Honey) with confidence 0.92, identifying attributes: raw, forest, pure

Input: "Organic Tamarind Paste 400g"

Keyword classifier: Matches "tamarind" → confidence 0.55
LLM classifier: Matches MFP #1 (Tamarind) with confidence 0.95, noting it's a processed current product

Input: "Wild Honey Face Wash - Natural Skin Care"

Keyword classifier: Matches "honey", "wild" → confidence 0.7 (false positive)
LLM classifier: Matches with confidence 0.3, correctly identifying this as a cosmetic product that uses honey as an ingredient, not a direct MFP product

Input: "Assorted Dry Fruits Gift Box"

Keyword classifier: No match
LLM classifier: Correctly returns empty `matched_mfp_ids` — this is a curated gift box, not an MFP product

8.4 Confidence Scoring

The classification pipeline produces confidence scores at two levels:

- Product-level confidence** (per scraped product): How likely this product belongs to a specific MFP category. Range [0.0, 1.0].
 - Keyword match alone: 0.4–0.7
 - LLM classification: 0.5–0.95
 - Combined (keyword + LLM agreement): max of both scores
- Field-level confidence** (per MFP analysis): How reliable the overall market analysis is, given the data quality. Computed in `calculate_confidence()`:
 - `product_count`: More products → higher confidence (capped at 20 products = 1.0)

- **pricing**: Percentage of products with valid prices
- **demand_score**: Based on review data availability (capped at 10 products with reviews = 1.0)
- **trend**: Average of the above three confidence scores

9. Data Quality & Reliability

9.1 Validation Mechanisms

The `market_validator.py` module implements multi-layered validation:

Layer	Check	Severity
Schema	Required fields present (status, market_products, market_summary, etc.)	Error
Product	Non-empty title, valid price (≥ 0), well-formed URL, rating $\in [0, 5]$	Error/Warning
Confidence	All confidence scores $\in [0, 1]$	Warning
Consistency	No duplicate <code>product_id</code> values	Warning
Completeness	Per-item completeness score (filled fields / total fields)	Metric
Coverage	Aggregate metrics: items with demand scores, pricing, competitor analysis	Metric

9.2 Error Handling Strategy

The pipeline implements a **fail-fast for single items, resilient for batches** strategy:

```
Single item (--item): Exception → raise immediately → user sees error
Batch mode (--all):   Exception → log to manifest.errors → continue to next item
```

This ensures that:

- During development, errors surface immediately for debugging
- During production batch runs, one problematic MFP doesn't abort the entire 87-item pipeline

9.3 Retry Logic

LLM calls implement **exponential backoff with provider-aware retry**:

1. **HTTP 429 (rate limit)**: Parse `Retry-After` header, sleep for specified duration + 1 second buffer
2. **Quota exhaustion**: Add model to `exhausted_models` set, immediately skip to next model
3. **Transient errors**: Retry up to `LLM_MAX_RETRIES` times with $3 \times (\text{attempt} + 1)$ second backoff
4. **Parse failures**: Attempt JSON extraction with fallback patterns (strip markdown fences, find `{...}` boundaries)

9.4 Product Deduplication

Products are deduplicated at two levels:

- 1. **Within a single MFP:** `fetch_products_for_mfp()` tracks `seen_ids` across all queries, preventing the same product from appearing multiple times when different queries return overlapping results
- 2. **Across MFPs:** `market_products.json` uses the global `product_index` dict keyed by `product_id`. When a product matches multiple MFPs, its `matched_mfp_ids` array is extended, and the maximum confidence is retained

The deterministic `make_product_id(url, title, price)` function ensures that the same product always receives the same ID, regardless of which query discovered it or when it was scraped.

9.5 Noise Reduction

Three mechanisms reduce noise in the product dataset:

- 1. **Keyword pre-filter:** Products that match zero keywords receive a baseline confidence of 0.0, making them unlikely to pass classification
- 2. **LLM confidence gating:** The LLM prompt explicitly instructs conservative matching — "only match when confident"
- 3. **MFP ID filtering:** After classification, only products with the target MFP ID in their `matched_mfp_ids` contribute to that MFP's market summary

10. Scalability Analysis

10.1 Current Scale: 87 MFPs

Metric	Value
Serper API calls	435 (5 queries × 87)
LLM classification calls	87 (1 batch per MFP)
LLM analysis calls	87 (1 per MFP)
Total API calls	~609
Estimated runtime	45–75 minutes
Serper cost	\$0.44
LLM cost (Gemini)	~\$0.03
Total cost per run	~\$0.50

10.2 Projected Scale: 500 MFPs

Metric	Value
Serper API calls	2,500
LLM calls	1,000

Metric	Value
Estimated runtime	4–6 hours
Serper cost	\$2.50
LLM cost (Gemini)	~\$0.15
Total cost per run	~\$2.65

Bottleneck: Sequential LLM processing with 4-second rate limits. At 500 MFPs, the 4-second LLM delay alone accounts for ~67 minutes.

Optimization: Implement async processing with connection pooling and parallel LLM requests (up to provider rate limits).

10.3 Projected Scale: 5,000 MFPs

Metric	Value
Serper API calls	25,000
LLM calls	10,000
Estimated runtime	40–60 hours (sequential)
Serper cost	\$25.00
LLM cost (Gemini)	~\$1.50
Total cost per run	~\$26.50

Bottleneck: At this scale, the pipeline requires fundamental architectural changes:

- 1. **Async I/O:** Replace `requests` with `aiohttp` for concurrent API calls
- 2. **Worker pool:** Distribute MFP processing across multiple workers
- 3. **Caching layer:** Cache Serper results with TTL (e.g., Redis) to avoid redundant queries on re-runs
- 4. **Incremental processing:** Only re-process MFPs whose market data is older than a configurable threshold
- 5. **Batch LLM classification:** Group products from multiple MFPs into larger batches for classification, reducing total LLM call count by 5–10×

10.4 Optimization Strategies Summary

Strategy	Impact	Complexity	Priority
Async HTTP requests	3–5× runtime reduction	Medium	High
LLM batch classification	5–10× fewer LLM calls	Low	High
Serper result caching	Eliminate redundant queries	Medium	Medium
Incremental processing	Skip unchanged items	Low	Medium
Worker pool parallelism	Linear scaling with workers	High	Low (needed at 1000+ MFPs)

11. Risks and Limitations

11.1 Marketplace Bias

Risk: Google Shopping results are biased toward established platforms (Amazon, Flipkart) and well-funded sellers. Niche MFP products sold through tribal cooperative websites, regional e-commerce platforms (e.g., Tribes India), or social commerce channels (WhatsApp, Instagram) are systematically underrepresented.

Impact: Demand scores may underestimate true market demand for MFPs with strong informal or direct-to-consumer channels.

Mitigation: Future versions should incorporate Tribes India product data and social media signal analysis.

11.2 Review Manipulation

Risk: Incentivized reviews, fake reviews, and review farms inflate review counts and ratings for specific products, particularly on Amazon India.

Impact: Products with manipulated reviews may artificially inflate demand scores for their associated MFP categories.

Mitigation: The logarithmic normalization of review counts dampens the impact of extreme outliers. Additionally, seller diversity serves as a cross-check — manipulated reviews typically concentrate on single-seller products.

11.3 Incomplete Coverage

Risk: Some MFP commodities (e.g., Puwad Seed, Baheda, Hill Broom) have minimal e-commerce presence, yielding few or zero search results.

Impact: These items receive near-zero demand scores, which may misrepresent their actual market potential in traditional/offline channels.

Mitigation: Items with fewer than 5 matched products receive low confidence scores, flagging them for manual review.

11.4 Missing Sales Data

Risk: No e-commerce platform exposes actual unit sales or revenue data. All demand estimation is proxy-based.

Impact: The demand score is a relative ranking metric, not an absolute measure of market size.

Mitigation: Clearly documented in output schemas — the `demand_score` field is labeled as a proxy score, not a sales estimate.

11.5 LLM Classification Errors

Risk: LLM misclassification (false positives or false negatives) can distort market analysis.

Impact: A false positive (cosmetic product classified as Wild Honey) inflates product counts and distorts pricing. A false negative (missed honey product) reduces coverage.

Mitigation: Dual classification (keyword + LLM) with confidence scoring. Products with confidence below 0.5 contribute less weight to aggregated metrics.

11.6 Price Volatility

Risk: E-commerce prices fluctuate daily based on promotions, inventory, and algorithmic pricing. A single snapshot may capture anomalous pricing.

Impact: Average price and price range may not reflect stable market conditions.

Mitigation: Future versions should implement multi-snapshot averaging (e.g., weekly runs with rolling averages).

12. Future Improvements

12.1 Short-Term (Next 2 Sprints)

Improvement	Impact	Effort
Percentile-based trend classification	More statistically meaningful trend labels	Low
Adaptive query expansion	Better coverage for niche MFPs	Low
Recommended weight update (35/25/15/15/10)	More accurate demand ranking	Trivial
Batch LLM classification	5-10× fewer API calls	Medium

12.2 Medium-Term (Next Quarter)

Improvement	Impact	Effort
DataForSEO integration	Historical pricing and trend data	Medium
Tribes India product catalog	Direct tribal cooperative data source	Medium
Async pipeline	3–5× runtime improvement	Medium
Time-series data collection	Enable genuine temporal trend analysis	High

12.3 Long-Term (Next 6 Months)

Improvement	Impact	Effort
Demand prediction models	Forecast future demand using ARIMA/Prophet	High
Recommendation engine integration	Feed demand data into Module 6	High
Fine-tuned classification model	Custom MFP classifier fine-tuning	High

Improvement	Impact	Effort
Reinforcement learning	Optimize search queries based on classification feedback	Very High
Product opportunity scoring	Identify untapped product-MFP combinations	Medium
Regional demand segmentation	State-level and district-level demand analysis	High
Sentiment analysis	Extract qualitative demand signals from reviews	Medium

13. Conclusion

Technical Assessment

Module 2 of the MAERII platform successfully addresses the **information asymmetry** in India's MFP sector by constructing a systematic, reproducible, and extensible market intelligence pipeline. The engineering decisions reflect a deliberate prioritization of **reliability over complexity**, **cost efficiency over feature richness**, and **extensibility over premature optimization**.

Architectural Strengths

- Proven pattern inheritance:** By mirroring Module 1's pipeline architecture (Search → Fetch → Extract → Validate → Write), Module 2 benefits from battle-tested patterns for atomic persistence, provider fallback, and incremental processing.
- Separation of concerns:** The clean layering — data models (pure functions) → service clients (API abstraction) → pipeline orchestrator (state management) → CLI (zero logic) — enables independent testing, modification, and replacement of each component.
- Graceful degradation:** The system operates meaningfully even when components fail: no LLM → keyword classification only; no Serper → web search fallback; no API keys → informative error messages.
- Reproducibility:** Deterministic product IDs, run manifests, backup snapshots, and preview mode enable full auditability and rollback capability.

Known Limitations

The most significant limitation is the **cross-sectional nature** of the analysis — the pipeline captures a market snapshot, not a temporal trend. The "trend" classification is more accurately described as "engagement level classification." True trend analysis requires longitudinal data collection, which is architecturally supported (via run manifests and incremental merging) but not yet implemented.

Final Recommendation

Module 2 is **production-ready** for its intended scope: generating structured market intelligence for 87 MFP commodities at minimal cost (~\$0.50 per full run). The demand scores, competitor analyses, and market summaries provide actionable insights for the downstream modules of the MAERII platform. The identified improvements (percentile ranking, weight optimization, async processing, historical data collection) represent a clear, prioritized roadmap for continuous enhancement without requiring architectural redesign.

Document Version: 1.0 | Last Updated: June 2026 | Next Review: After Module 3 integration